



YOHPAL LIVE RELEASE 1.0G-RC1 — Validation Report

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Sat, 11 Jul at 12:30

YOHPAL LIVE RELEASE 1.0G-RC1 — Validation Report

The latest verified Release 1.0F report confirms that the codebase has advanced materially, but it also records **no commit, no tag, no Android/iPhone device run, and no completed 1.0G evidence package**. It further identifies Crashlytics as instrumented but unverified, environment abstraction as partial, and rewarded-ad verification as still incomplete.

report F-G .pdf

Accordingly, this validation report must remain evidence-driven.

Opening Decision

Current decision: NO-GO

This decision can change to **CONDITIONAL GO** or **GO** only after developers submit the tagged RC1 commit and the complete Android/iPhone evidence package.

1. Release Candidate Identity

Evidence	Required value	Status
Release branch	release/yohpal-live-1.0g-rc1 or approved equivalent	Pending
Commit hash	Full immutable SHA	Pending
RC1 tag	Annotated Git tag	Pending
Clean working tree	No uncommitted changes	Pending
CI run	Must reference the same commit	Pending

A certificate must never be issued against an uncommitted working tree.

2. Critical Closure Matrix

Critical finding	Required closure evidence	Status
Analytics route crash	Automated route test and device proof	Source-fixed; device proof pending
Crash reporting	Android and iPhone Crashlytics events	Pending

Password reset	Email delivery and completed reset on devices	Source-fixed; device proof pending
Environment configuration	All backend/web endpoints supplied through environment config	Pending final proof
Rewarded-ad fraud exposure	Real ad SDK and cryptographically verified server callback	Pending

Critical Gate Rule

```
bool criticalGatePasses(List<CertificationFinding> findings) {
    return findings
        .where((finding) => finding.severity == Severity.critical)
        .every((finding) => finding.status == FindingStatus.passed);
}
```

No Critical finding may remain:

- open,
- partial,
- mitigated,
- source-only,
- or unverified.

3. High Closure Matrix

The Release 1.0F report shows **7 of 13 High findings fixed**, two partially addressed, and four untouched.
report F-G .pdf

High finding	Required proof	Status
Search navigation	Android/iPhone journey	Pending
Live Discovery navigation	Android/iPhone journey	Pending
Feed retry	Forced network error test	Pending
Accessibility semantics	TalkBack and VoiceOver proof	Pending
Live reconnect	Actual network interruption proof	Pending
Duplicate video stacks	Commit diff and analyzer proof	Source-verified
Friendly auth errors	Device screenshots	Pending
Input validation	Login and signup validation tests	Pending
Automated test coverage	Coverage report	Pending
AI publish gate	End-to-end gate proof	Pending

Avatar upload	Storage upload proof	Pending
Chat error surfacing	Failed-send retry proof	Pending
Logout state clearing	Second-account isolation proof	Pending

Recommended High Threshold

For GO:

0 open High findings

For CONDITIONAL GO:

Maximum 2 non-financial High findings open
Each must:

- have a named owner,
- have a deadline,
- be feature-flagged or bounded,
- not affect authentication, money, security, privacy, or data integrity.

4. Required Android Validation

Developers must return evidence for:

- ANDROID-RC1-01 Release installation
- ANDROID-RC1-02 Email login
- ANDROID-RC1-03 Google Sign-In
- ANDROID-RC1-04 Password reset
- ANDROID-RC1-05 Logout
- ANDROID-RC1-06 Second-account isolation
- ANDROID-RC1-07 Search
- ANDROID-RC1-08 Live Discovery
- ANDROID-RC1-09 Feed retry
- ANDROID-RC1-10 Feed pagination
- ANDROID-RC1-11 Mute/unmute
- ANDROID-RC1-12 Double-tap like
- ANDROID-RC1-13 Live reconnect
- ANDROID-RC1-14 Wallet return
- ANDROID-RC1-15 Rewarded-ad verification
- ANDROID-RC1-16 Avatar upload
- ANDROID-RC1-17 Chat failed-send retry
- ANDROID-RC1-18 TalkBack accessibility
- ANDROID-RC1-19 Crashlytics non-fatal
- ANDROID-RC1-20 Crashlytics fatal
- ANDROID-RC1-21 30-minute feed playback
- ANDROID-RC1-22 30-minute live session

Each result must include:

- device model,
- Android version,
- app build number,
- commit hash,
- timestamp,
- tester,
- outcome,
- screenshot/video,
- relevant logs.

5. Required iPhone Validation

- IOS-RC1-01 Release installation
- IOS-RC1-02 Email login
- IOS-RC1-03 Google Sign-In
- IOS-RC1-04 Apple Sign-In
- IOS-RC1-05 Password reset
- IOS-RC1-06 Logout
- IOS-RC1-07 Second-account isolation
- IOS-RC1-08 Search
- IOS-RC1-09 Live Discovery
- IOS-RC1-10 Feed retry

```
IOS-RC1-11  Pagination
IOS-RC1-12  Mute
IOS-RC1-13  Double-tap like
IOS-RC1-14  Live reconnect
IOS-RC1-15  Wallet deep-link return
IOS-RC1-16  Rewarded-ad verification
IOS-RC1-17  Avatar upload
IOS-RC1-18  Chat failed-send retry
IOS-RC1-19  VoiceOver accessibility
IOS-RC1-20  Crashlytics non-fatal
IOS-RC1-21  Crashlytics fatal
IOS-RC1-22  30-minute feed playback
IOS-RC1-23  30-minute live session
```

A physical iPhone is required for final certification.

6. Required Codebase — Certification Models

tool/release_certification/certification_finding.dart

```
enum Severity {
  critical,
  high,
  medium,
  low,
}

enum FindingStatus {
  pending,
  passed,
  failed,
  acceptedRisk,
}

final class CertificationFinding {
  const CertificationFinding({
    required this.id,
    required this.title,
    required this.severity,
    required this.status,
    required this.evidence,
    this.owner,
    this.deadline,
    this.notes,
  });

  final String id;
  final String title;
  final Severity severity;
  final FindingStatus status;
  final List<String> evidence;
  final String? owner;
  final DateTime? deadline;
  final String? notes;

  bool get isBlocking {
    if (severity == Severity.critical) {
      return status != FindingStatus.passed;
    }

    if (severity == Severity.high) {
      return status == FindingStatus.failed ||
        status == FindingStatus.pending;
    }

    return false;
  }
}
```

tool/release_certification/release_decision_engine.dart

```
enum ReleaseDecision {
  go,
  conditionalGo,
  noGo,
}

final class ReleaseDecisionInput {
  const ReleaseDecisionInput({
    required this.findings,
    required this.androidPassed,
    required this.iosPassed,
  });
}
```

```

        required this.crashlyticsPassed,
        required this.securityPassed,
        required this.ciPassed,
        required this.rollbackPassed,
    });

    final List<CertificationFinding> findings;
    final bool androidPassed;
    final bool iosPassed;
    final bool crashlyticsPassed;
    final bool securityPassed;
    final bool ciPassed;
    final bool rollbackPassed;
}

ReleaseDecision decideRelease(ReleaseDecisionInput input) {
    final criticalNotPassed = input.findings.any(
        (finding) =>
            finding.severity == Severity.critical &&
            finding.status != FindingStatus.passed,
    );

    if (criticalNotPassed ||
        !input.androidPassed ||
        !input.iosPassed ||
        !input.crashlyticsPassed ||
        !input.securityPassed ||
        !input.ciPassed ||
        !input.rollbackPassed) {
        return ReleaseDecision.noGo;
    }

    final openHigh = input.findings.where(
        (finding) =>
            finding.severity == Severity.high &&
            finding.status != FindingStatus.passed,
    );

    if (openHigh.isEmpty) {
        return ReleaseDecision.go;
    }

    final conditionallyAcceptable = openHigh.length <= 2 &&
        openHigh.every(
            (finding) =>
                finding.status == FindingStatus.acceptedRisk &&
                finding.owner != null &&
                finding.deadline != null,
        );

    return conditionallyAcceptable
        ? ReleaseDecision.conditionalGo
        : ReleaseDecision.noGo;
}

```

7. Device Evidence Schema

release-evidence/yohpal-live-1.0g-rc1/device-evidence.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "YohPal Live Device Evidence",
  "type": "object",
  "required": [
    "testId",
    "platform",
    "deviceModel",
    "osVersion",
    "buildNumber",
    "commitHash",
    "status",
    "tester",
    "executedAt",
    "evidencePaths"
  ],
  "properties": {
    "testId": {
      "type": "string"
    },
    "platform": {
      "enum": ["android", "ios"]
    },
    "deviceModel": {

```

```

        "type": "string"
    },
    "osVersion": {
        "type": "string"
    },
    "buildNumber": {
        "type": "string"
    },
    "commitHash": {
        "type": "string",
        "minLength": 40,
        "maxLength": 40
    },
    "status": {
        "enum": ["passed", "failed", "blocked"]
    },
    "tester": {
        "type": "string"
    },
    "executedAt": {
        "type": "string",
        "format": "date-time"
    },
    "evidencePaths": {
        "type": "array",
        "minItems": 1,
        "items": {
            "type": "string"
        }
    },
    "notes": {
        "type": "string"
    }
}
}
}

```

8. Evidence Validation Script

tool/release_certification/validate_rcl_evidence.dart

```

import 'dart:convert';
import 'dart:io';

void main(List<String> args) {
  if (args.length != 1) {
    stderr.writeln(
      'Usage: dart run tool/release_certification/'
      'validate_rcl_evidence.dart <evidence-root>',
    );
    exitCode = 64;
    return;
  }

  final root = Directory(args.first);

  if (!root.existsSync()) {
    stderr.writeln('Evidence directory does not exist.');
```

```

    exitCode = 66;
    return;
  }

  final manifestFile = File('${root.path}/manifest.json');
```

```

  if (!manifestFile.existsSync()) {
    stderr.writeln('manifest.json is missing.');
```

```

    exitCode = 1;
    return;
  }

  final manifest = jsonDecode(
    manifestFile.readAsStringSync(),
  ) as Map<String, dynamic>;

  final requiredFields = <String>[
    'release',
    'branch',
    'commitHash',
    'tag',
    'ciRun',
    'androidReport',
    'iosReport',
    'crashlyticsProof',
    'securityReport',
  ];

```

```

    'rrbResolution',
  ];

  final missingFields = requiredFields.where(
    (field) =>
      !manifest.containsKey(field) ||
      manifest[field] == null ||
      manifest[field].toString().trim().isEmpty,
  );

  if (missingFields.isNotEmpty) {
    stderr.writeln(
      'Manifest fields missing: ${missingFields.join(', ')}',
    );
    exitCode = 2;
    return;
  }

  final referencedPaths = <String>[
    manifest['androidReport'] as String,
    manifest['iosReport'] as String,
    manifest['crashlyticsProof'] as String,
    manifest['securityReport'] as String,
    manifest['rrbResolution'] as String,
  ];

  final missingPaths = referencedPaths.where(
    (path) =>
      !File(path).existsSync() &&
      !Directory(path).existsSync(),
  );

  if (missingPaths.isNotEmpty) {
    stderr.writeln('Missing referenced evidence:');
    for (final path in missingPaths) {
      stderr.writeln(' - $path');
    }

    exitCode = 3;
    return;
  }

  stdout.writeln(
    'RC1 evidence structure is complete for '
    '${manifest['commitHash']}',
  );
}

```

9. RC1 Manifest

release-evidence/yohpal-live-1.0g-rc1/manifest.json

```

{
  "release": "YohPal Live 1.0G-RC1",
  "branch": "release/yohpal-live-1.0g-rc1",
  "commitHash": "REPLACE_WITH_FULL_COMMIT_HASH",
  "tag": "yohpal-live-v1.0.0-rc1",
  "ciRun": "REPLACE_WITH_CI_RUN_REFERENCE",
  "androidReport": "release-evidence/yohpal-live-1.0g-rc1/android/report.json",
  "iosReport": "release-evidence/yohpal-live-1.0g-rc1/ios/report.json",
  "crashlyticsProof": "release-evidence/yohpal-live-1.0g-rc1/observability/",
  "securityReport": "release-evidence/yohpal-live-1.0g-rc1/security/security-report.md",
  "rrbResolution": "release-evidence/yohpal-live-1.0g-rc1/governance/rrb-resolution.md"
}

```

10. Validation Commands

```

git status --short
git branch --show-current
git rev-parse HEAD
git tag --points-at HEAD

flutter clean
flutter pub get
dart format --output=none --set-exit-if-changed lib test tool
flutter analyze
dart run tool/check_routes.dart
flutter test --coverage

cd functions

```

```
npm ci
npm run build
npm test
```

Then:

```
dart run \
  tool/release_certification/validate_rc1_evidence.dart \
  release-evidence/yohpal-live-1.0g-rc1
```

11. Release Review Board Resolution

Decision options

GO

Issue only if:

- all Critical findings pass,
- all High findings pass,
- Android passes,
- iPhone passes,
- Crashlytics is proven,
- rewarded-ad server verification passes,
- CI passes against tagged commit,
- rollback proof exists.

CONDITIONAL GO

Issue only if:

- all Critical findings pass,
- a maximum of two non-financial High findings remain,
- both are formally accepted,
- each has an owner and deadline,
- rollout remains at 1–5%,
- Mission Control and kill switches are active.

NO-GO

Mandatory where:

- any Critical item remains partial or unverified,
 - either device report is incomplete,
 - Crashlytics is not visible in Firebase,
 - SSO or password reset fails,
 - ad rewards are not server verified,
 - evidence does not match the tagged commit.
-

Current Formal Decision

Release 1.0G-RC1

NO-GO

Basis

The evidence currently available is source-level Release 1.0F verification only. It explicitly states that:

- work remained uncommitted,
- no tag existed,
- no physical device testing occurred,
- Crashlytics was not verified,
- environment configuration remained partial,
- rewarded-ad verification remained incomplete.

report F-G .pdf

This decision should remain unchanged until the developers submit the actual RC1 evidence package.

Next Smart Move

Require developers to submit the **tagged RC1 Evidence Pack** against one immutable commit. Once submitted, perform the Release Review Board review and issue the final **GO**, **CONDITIONAL GO**, or **NO-GO** decision—without opening another feature or implementation release.